

Interpreters as Natural transformations

Luca Roversi

Learning outcomes

- A “natural” pattern for our interpreter(s)
- Natural transformations among functors
- Examples of Natural transformations in Haskell
- `nat :: Term a -> Result a` is natural transformationZ
- Strong motivations to have natural interpreters
- Exercises

A “natural” pattern of interpreters for any `Num` a

- Interpreter for `Const` only
 - from `Arithmetic.Interpreters.Functorial.Const.Eval.hs`
 - to `Arithmetic.Interpreters.Natural.Const.Eval.hs`
- Interpreter for `Add`
 - from `Arithmetic.Interpreters.Functorial.Add.Eval.hs`
 - to `Arithmetic.Interpreters.Natural.Add.Eval.hs`
- Interpreter for `Add` and `Mul`
 - from `Arithmetic.Interpreters.Functorial.Mul.Eval.hs`
 - to `Arithmetic.Interpreters.Natural.Mul.Eval.hs`

Natural Transformation

Assumption.

$\mathcal{C} \begin{array}{c} \xrightarrow{F} \\ \xleftarrow{G} \end{array} \mathcal{D}$ functors with the same domain and codomain.

Natural transformation from F to G .

Family $\{\eta_a : F \rightarrow G\}_{a \in \mathcal{C}}$ s.t. the red “box”:

$$\begin{array}{ccc} a & \xrightarrow{f} & b \\ \downarrow F & & \downarrow F \\ F(a) & \xrightarrow{F(f)} & F(b) \\ \downarrow \eta_a & & \downarrow \eta_b \\ G(a) & \xrightarrow{G(f)} & G(b) \end{array}$$

The diagram illustrates the naturality condition for a natural transformation η from F to G . It shows a commutative square (the red box) with vertices $F(a)$, $F(b)$, $G(a)$, and $G(b)$. The top horizontal arrow is $F(f)$, the bottom horizontal arrow is $G(f)$, the left vertical arrow is η_a , and the right vertical arrow is η_b . The square is enclosed in a dashed circle labeled G on both sides. The top horizontal arrow f is labeled f above it. The vertical arrows from a and b to $F(a)$ and $F(b)$ are labeled F to the left and right respectively.

commutes for every morphism $f : a \rightarrow b$ in $\mathcal{C}$, i.e.:

$$\eta_b \circ F(f) = G(f) \circ \eta_a$$

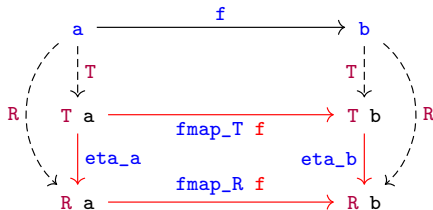
Examples: Natural Transformations in Haskell

- `Support.CT0450NaturalTransformations.hs`

`eta :: Term a -> Result a` is natural

Let rename `Term` as `T` and `Result` as `R`.

For every `f :: a -> b`, the diagram:



commutes.

Proof (just to have a look ...).

`Arithmetic.Interpreters.Natural.EtaTermToResultIsNatural.hs`

Exercise?!?

Extend the proof in

`Arithmetic.Interpreters.Natural.EtaTermToResultIsNatural.hs`

to the case `Tmul`

Nice that `eta :: Term s --> Result b` is natural

Compiler optimizations are harmless w.r.t. natural interpreters

Naturality ensures that an interpreter is stable under:

- α -renaming
- Constant folding
- Dead-code elimination
- Changing types or runtime representations

Example: changing runtime representations.

At `Arithmetic.Interpreters.Natural`:

- `JustificationFirst.EtaFmapBetterThanFmapEta.hs`
- `Main.hs` (to be put in `JustificationFirst`.)
- `JustificationSecond.EtaFmapBetterThanFmapEta.hs`
- `Main.hs` (to be put in `JustificationFirst`.)

Nice that $\text{eta} :: \text{Term } s \rightarrow \text{Result } b$ is natural

- Being natural is the minimal requirement for an interpreter that must be step-wise and modularly extended
 - adding new constructors (negation, exponentiation, comparisons...)
 - adding side effects (logging, tracing, exceptions)
- Without naturality, extension:
 - fails or
 - becomes ad *hoc*

Exercises

New interpreters “for free”

- An interpreter ‘`Term (Set a) -> Result (Set a)`’
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Structure.hs`
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Operations.hs`
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise1.hs`
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise1_SOLUTION.hs` (restricted)
- Interpreter from ‘`Set a`’ to ‘`Result a`’ between monoids
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise2.hs`
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise2_SOLUTION.hs` (restricted)
- Instances of the interpreter from ‘`Set a`’ to ‘`Result a`’
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise2dot1.hs`
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise2dot1_SOLUTION.hs` (restricted)
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise2dot2.hs`
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise2dot2_SOLUTION.hs` (restricted)
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise2dot3.hs`
 - `Arithmetic.Interpreters.Natural.Exercises.Set.Exercise2dot3_SOLUTION.hs` (restricted)